Maths for ML Handbook

Equations and Formulas used in Machine Learning

Version: 2.0

Index

Linear Algebra

Differential Calculus

1 Linear Equation

1.1 Slope-Intercept Equation

Slope-Intercept Form

$$y = m \cdot x \ + \ c$$

Where,

- m - Slope (angle between straight line and x-axis)
- c - intercept (distance of straight line from the origin)
- x - Independent variable
- y - Dependent variable

Slope

Given two points (x_1, y_1) and (x_2, y_2), formula to calculate Slope m:

$$m = \frac{y_2 - y_1}{x_2 - x_1}$$

Intercept

Given two points (x_1, y_1) and (x_2, y_2), formula to calculate Intercept c:

$$c = y_1 - (m \cdot x_1)$$

$$\text{or}$$

$$c = y_2 - (m \cdot x_2)$$

1.2 Limitations of Slope-Intercept Equation

Limitations

1. Slope m becomes undefined or ∞ when $x_2 - x_1 = 0$.
2. Not scalable for higher dimensions.

Solution

General Line Equation:

$$ax + by + c = 0$$

1.3 Geometric Relationship b/w lines

Parallel Lines

When given two lines $y = m_1x + c_1$ and $y = m_2x + c_2$ are parallel:

$$m_1 = m_2$$

Perpendicular Lines (Orthogonality)

When given two lines $y = m_1x + c_1$ and $y = m_2x + c_2$ are perpendicular:

$$m_1 \times m_2 = -1$$

2 General Line Equation

2.1 Standard Form

$$ax + by + c = 0$$

Rearranging above General Line Equation in Slope-Intercept form:

$$w_1x + w_2y + w_0 = 0$$

$$w_2y = -w_1x - w_0$$

$$y = \left(-\frac{w_1}{w_2}\right)x + \left(-\frac{w_0}{w_2}\right)$$

Where Slope and Intercept can be derived as,

$$m = -\frac{w_1}{w_2} \qquad c = -\frac{w_0}{w_2}$$

2.2 Advantages

1. Scaling higher dimensions is achieved by Linear Expansion of the equation.
2. Undefined slope is solved by setting the coefficient of y i.e., b in GLE: $ax + by + c = 0$ to zero.

2.3 Scaling

Line Equation (2D Hyperplane)

General Line Equation for 2 dimensional data:

$$w_1x_1 + w_2x_2 + w_0 = 0$$

Plane Equation (3D Hyperplane)

Scaling General Line Equation for 3 dimensional data:

$$w_1x_1 + w_2x_2 + w_3x_3 + w_0 = 0$$

nD Hyperplane Equation

Scaling General Line Equation for higher dimensional ($d > 3$) data:

$$w_1x_1 + w_2x_2 + \ldots + w_nx_n + w_0 = 0$$

3 Vector

3.1 Vector Representation

Mathematical notation of a vector $\vec{x}$ having d dimension containing real numbers:

$$x \in \mathbb{R}^d$$

Where d represents number of elements in the vector.

Row Vector

$$\vec{x} = [x_1\ x_2\ \ldots\ x_d]$$

Column Vector

$$\vec{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_d \end{bmatrix}$$

3.2 Types of Distances

Various methods Calculate distance between two points (x_1, y_1) and (x_2, y_2).

1 Euclidean Distance

Formula to calculate Euclidean Distance between two points (x_1, y_1) and (x_2, y_2)

$$d = \sqrt{(y_2 - y_1)^2 + (x_2 - x1)^2}$$

2 Manhattan Distance

Formula to calculate Manhattan Distance between two points (x_1, y_1) and (x_2, y_2)

$$d = |y_2 - y_1| + |x_2 - x_1|$$

3.3 Types of Norms

Norm

Norm of a vector represents the straight-line distance from the origin to the endpoint of the vector.

L2 Norm of a Vector

Formula for L2 Norm or Euclidean Distance of a vector of d dimension:

$$\|\vec{x}\|_2 = \sqrt{{x_1}^2 + {x_2}^2 + {x_3}^2 + \ldots + {x_d}^2}$$

$$= \sqrt{\sum_{i=1}^{d} {x_i}^2}$$

L1 Norm of a Vector

Formula for L1 Norm or Manhattan Distance of a vector of d dimension:

$$\|\vec{x}\|_1 = |x_1| + |x_2| + |x_3| + \ldots + |x_d|$$

$$= \sum_{i=1}^{d} |x_i|$$

4 Matrix Multiplication

4.1 Matrix Multiplication rule

Given two matrices A and B where:

- $A : m \times n$
- $B : n \times k$

Rule

Number of columns in the first matrix should be equal to the number of rows in the second matrix.

$$m \times n \Leftrightarrow n \times k = m \times k$$

The resulting matrix will be of shape: $m \times k$

4.2 Matrix Multiplication in NumPy

Given two NumPy matrices `m1` and `m2`

```
m1 @ m2
# or
np.matmul(m1, m2)
```

5 Dot Product

5.1 Dot Product of Vectors

Formula to calculate dot product of two vectors $\vec{a}$ and $\vec{b}$:

$$\vec{a} \cdot \vec{b} = a^T \cdot b$$

$$= \sum_{i=1}^{d} a_i b_i \quad \in \mathbb{R}^1$$

5.2 Dot Product in NumPy

Given two NumPy vectors `v1` and `v2`

```
np.dot(v1, v2)
```

6 Application of Dot Product

6.1 Angle between Vectors

Formula to calculate angle between two vectors $\vec{a}$ and $\vec{b}$

$$\cos\theta = \frac{a^T \cdot b}{\|a\| \, \|b\|}$$

Cauchy-Schwarz Inequality

Absolute value of dot product of two vectors is always less than or equal to the product of their magnitudes.

$$\left| \vec{a} \cdot \vec{b} \right| \ \leq \ \|\vec{a}\| \, \|\vec{b}\|$$

Perpendicular Vectors

When two vectors are **perpendicular** to each other, their **dot product is zero**.

If two vectors are perpendicular to each other, then $\theta = 90^\circ$ and $\cos 90^\circ = 0$, therefore,

$$\vec{a} \cdot \vec{b} = 0$$

6.2 Machine Learning

In ML context, the dot product is used to represent the below "best fit" line or hyper-plane equation:

$$w_1 x_1 + w_2 x_2 + \ldots + w_d x_d + w_0 = 0$$

using $\vec{w}$ and $\vec{x}$ vectors of d dimensions:

$$\vec{w} = \begin{bmatrix} w_1 \\ w_2 \\ \vdots \\ w_d \end{bmatrix} \qquad \vec{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_d \end{bmatrix}$$

in **Linear Algebraic** form as,

$$w^T \cdot x \ + \ w_0 = 0$$

Where $w^T \cdot x$ is the dot product between weight vector $\vec{w}$ and feature vector $\vec{x}$.

6.3 Projection of a Vector

Unit Vector

Unit vector is defined as a vector whose magnitude is equal to 1.

$$\|\vec{a}\| = 1$$

Formula to calculate unit vector $\widehat{w}$ of a $\vec{w}$ vector:

$$\widehat{w} = \frac{\vec{w}}{\|w\|}$$

Scalar Projection

$$\text{scalar projection of } \vec{x} \text{ on } \vec{y} = \vec{x} \cdot \hat{y}$$

Formula to calculate scalar projection of $\vec{x}$ on $\vec{y}$:

$$\text{comp}_{\vec{y}} \vec{x} = \frac{\vec{x} \cdot \vec{y}}{\|\vec{y}\|}$$

Dot product is equal to the product of one vector and the projection of the vector on the first one.

$$\vec{a} \cdot \vec{b} = \|\vec{a}\| \times \left(\text{scalar projection of } \vec{b} \text{ on } \vec{a}\right)$$

or

$$\vec{a} \cdot \vec{b} = \|\vec{b}\| \times \left(\text{scalar projection of } \vec{a} \text{ on } \vec{b}\right)$$

Vector Projection

The Vector Projection of $\vec{x}$ onto $\vec{y}$ equals the Scalar Projection of $\vec{x}$ onto $\vec{y}$ times a unit vector in the direction of $\vec{y}$.

Formula to calculate vector projection of $\vec{x}$ on $\vec{y}$:

$$\begin{aligned}
\text{proj}_{\vec{y}} \vec{x} &= \left(\frac{\vec{x} \cdot \vec{y}}{\|y\|}\right) \hat{y} \\
&= \left(\frac{\vec{x} \cdot \vec{y}}{\|y\|}\right) \frac{\vec{y}}{\|y\|} \\
&= \frac{\vec{x} \cdot \vec{y}}{\|y\|^2} \vec{y}
\end{aligned}$$

7 Shifting Lines

7.1 One Direction

Given line equation:

$$w_1x_1 + w_2x_2 + w_0 = 0$$

Shifting lines in one direction (x-axis or y-axis) by a units:

$$\begin{aligned}
\text{x-axis}| \quad \text{Right} \rightarrow \quad & w_1(x_1 - a) + w_2x_2 + w_0 = 0 \\
\text{x-axis}| \quad \text{Left} \leftarrow \quad & w_1(x_1 + a) + w_2x_2 + w_0 = 0 \\
\text{y-axis}| \quad \text{Up} \uparrow \quad & w_1x_1 + w_2(x_2 - a) + w_0 = 0 \\
\text{y-axis}| \quad \text{Down} \downarrow \quad & w_1x_1 + w_2(x_2 + a) + w_0 = 0
\end{aligned}$$

7.2 Two Directions

Shifting lines in two direction (x-axis and y-axis) simultaneously by a units:

$$w_1(x_1 - a) + w_2(x_2 - a) + w_0 = 0$$

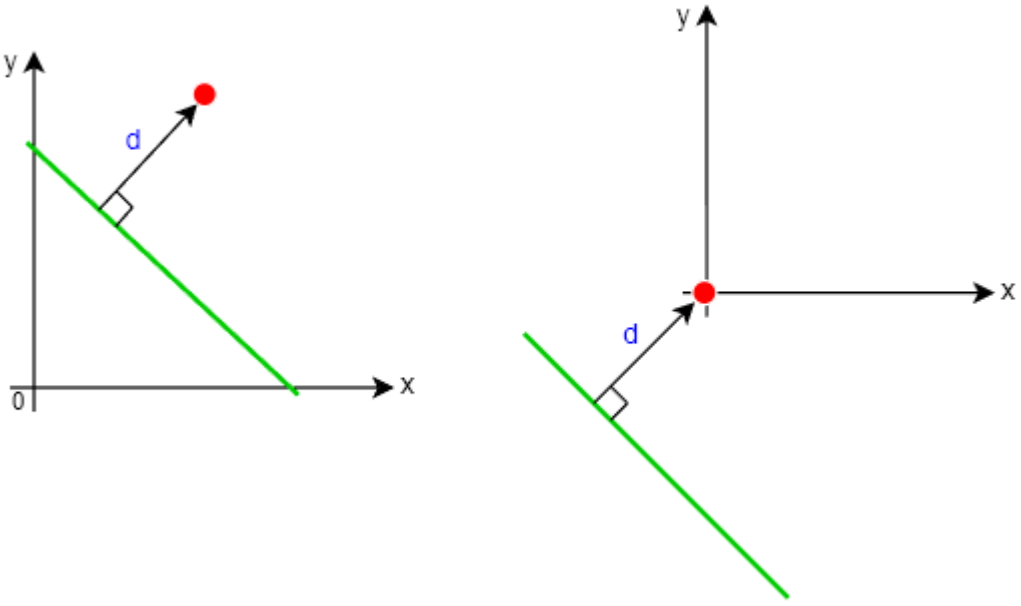
Solving above equation gives:

$$w_1x_1 + w_2x_2 + w'_0 = 0$$

Where w'_0 is:

$$w'_0 = w_0 - a(w1 + w2)$$

7.3 Usecase



8 Distances

8.1 Distance Between Line and Origin

Formula to calculate the distance between line (Decision Boundary) and origin using line's Normal vector $\vec{w}$

$$d = \frac{\left| -w_0 \right|}{\|w\|}$$

8.2 Distance Between Point and Line

Formula to calculate the distance between a point x_0 and line (Decision Boundary) using line's Normal vector $\vec{w}$

$$d = \frac{\left| w^T x_0 + w_0 \right|}{\|w\|}$$

8.3 Distance Between Parallel Lines

Same Normal Vector

Formula to calculate the distance between two parallel lines with Normal vector w having same magnitude:

$$d = \frac{\left| w_0^{(1)} - w_0^{(2)} \right|}{\|w\|}$$

Different Normal Vector

Formula to calculate the distance between two parallel lines with Normal vectors of different magnitudes where $w^{(1)} = \lambda \cdot w^{(2)}$

$$d = \left| \frac{w_0^{(1)}}{\|w^{(1)}\|} - \frac{w_0^{(2)}}{\|w^{(2)}\|} \right|$$

9 Linear Binary Classifier

9.1 Half-spaces

Positive Half-space

In Positive Half-space all data-points have **distance greater than zero** because,

$$w_1x_1 + w_2x_2 + \ldots + w_dx_d + w_0 > 0$$

Negative Half-space

In Negative Half-space all data-points have **distance less than zero** because,

$$w_1x_1 + w_2x_2 + \ldots + w_dx_d + w_0 < 0$$

Decision Boundary

On Decision Boundary all data-points have **distance equal to zero** because,

$$w_1x_1 + w_2x_2 + \ldots + w_dx_d + w_0 = 0$$

9.2 Mathematical Notation

Dataset

Equation to represent a labeled dataset D used in Linear Binary Classifier:

$$D := \left\{ \left(x_i, y_i\right) \; ; \; x \in \mathbb{R}^d \; ; \; y \in \{-1, +1\} \right\}_{i=1}^{n}$$

Where,

- D is Dataset
- x are features of datatype Real numbers $\mathbb{R}$
- y are labels
- n is the total number of rows in the dataset
- d is the total number of features/columns (dimension) in the dataset

Label

Actual Label y can take values:

$$y \begin{cases} +1 & \text{in } +\text{ve Half-Space} \\ \\ -1 & \text{in -ve Half-Space} \end{cases}$$

Classifier

Mapping function for a Linear Binary Classifier: nD Hyperplane

$$\begin{aligned}\hat{y} = f(x) &= w_1 x_1 + w_2 x_2 + \ldots + w_d x_d + w_0 \\ &\text{or} \\ &= w^T x + w_0\end{aligned}$$

Predicted label $\hat{y}$ can take values:

$$\hat{y} \begin{cases} +1 & \text{if } \left(w^T x + w_0\right) \geq 0 \\ -1 & \text{if } \left(w^T x + w_0\right) < 0 \end{cases}$$

9.3 Gain Function

Equation of Gain Function for Linear Binary Classifier:

$$g(D, \vec{w}, w_0) = \frac{1}{n} \sum_{i=1}^{n} \left(\frac{w^T x^{(i)} \ + \ w_0}{\|w\|} \right) \cdot y^{(i)}$$

9.4 Loss Function

Equation of Loss Function for Linear Binary Classifier:

$$\text{Loss Function} = - \text{ Gain Function}$$

$$l(D, \vec{w}, w_0) = -\frac{1}{n} \sum_{i=1}^{n} \left(\frac{w^T x^{(i)} \ + \ w_0}{\|w\|} \right) \cdot y^{(i)}$$

9.5 Perceptron Learning Algorithm

Weight Updation Logic

Logic in Perceptron Learning Algorithm to update Weights for miss-classified data-point:

$$\text{if } \ \text{sign}\left(\frac{w^T x^{(i)} + w_0}{\|w\|} \right) \ \neq \ \text{sign}\left(y^{(i)}\right) :$$

$$w = w + x^{(i)} \cdot y^{(i)}$$

$$w_0 = w_0 + y^{(i)}$$

10 Optimization

10.1 Optimization of Gain Function

Equation for Optimization of Gain Function for Linear Binary Classifier:

$$w^*, w_0^* = \arg\max_{w,w_0} \; g(D, \vec{w}, w_0)$$

$$= \arg\max_{w,w_0} \; \frac{1}{n} \sum_{i=1}^{n} \left(\frac{w^T x^{(i)} \; + \; w_0}{\|w\|} \right) \cdot y^{(i)}$$

Goal

Maximize Gain function.

10.2 Optimization of Loss Function

Equation for Optimization of Loss Function for Linear Binary Classifier:

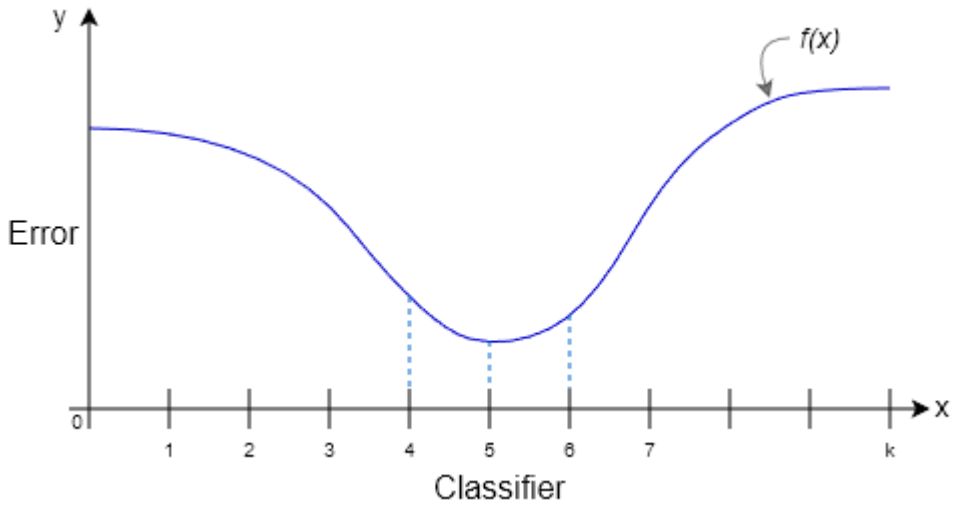
$$w^*, w_0^* = \arg\min_{w,w_0} \; l(D, \vec{w}, w_0)$$

$$= \arg\min_{w,w_0} \; -\frac{1}{n} \sum_{i=1}^{n} \left(\frac{w^T x^{(i)} \; + \; w_0}{\|w\|} \right) \cdot y^{(i)}$$

Goal

Minimize Loss function.

ML Context



Error vs Classifier is represented as a function $f(x)$ which needs to be minimized.

11 Functions

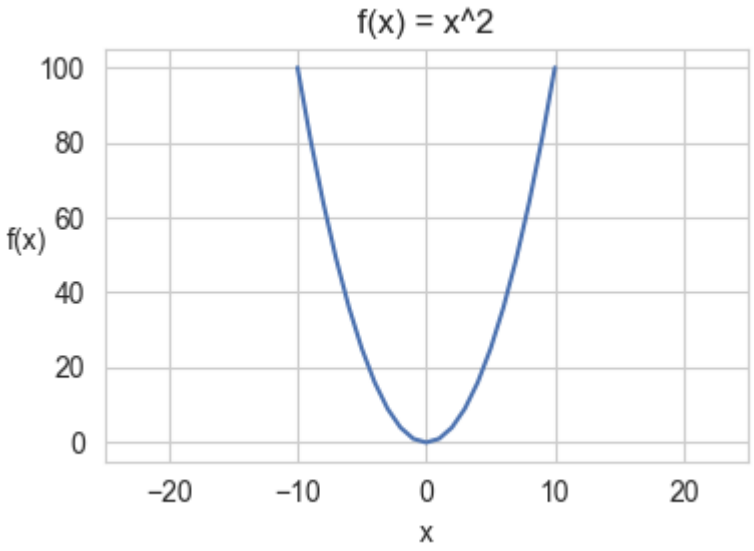
11.1 Continuous Function

Definition

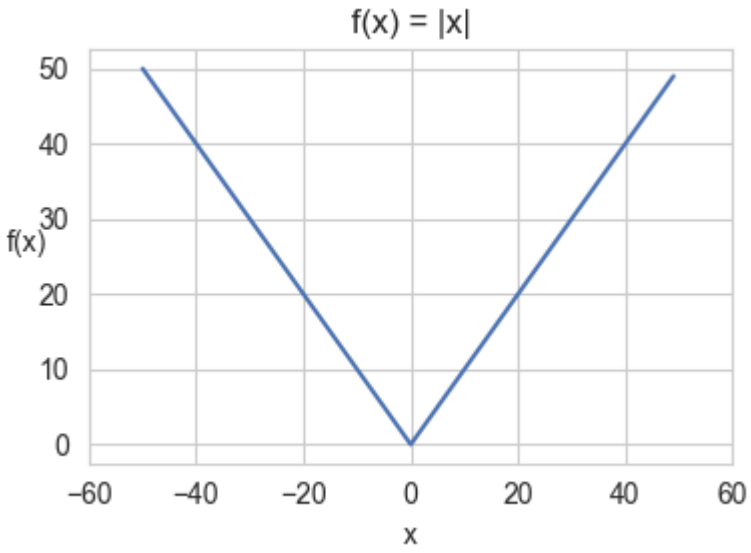
Criteria for a Continuous function:

$$\lim_{x \to a^-} f(x) = \lim_{x \to a^+} f(x) = f(a)$$

Example #1



Example #2



> **Note**: Even though $f(x) = |x|$ is continuous function, it is **not differentiable at** $x = 0$

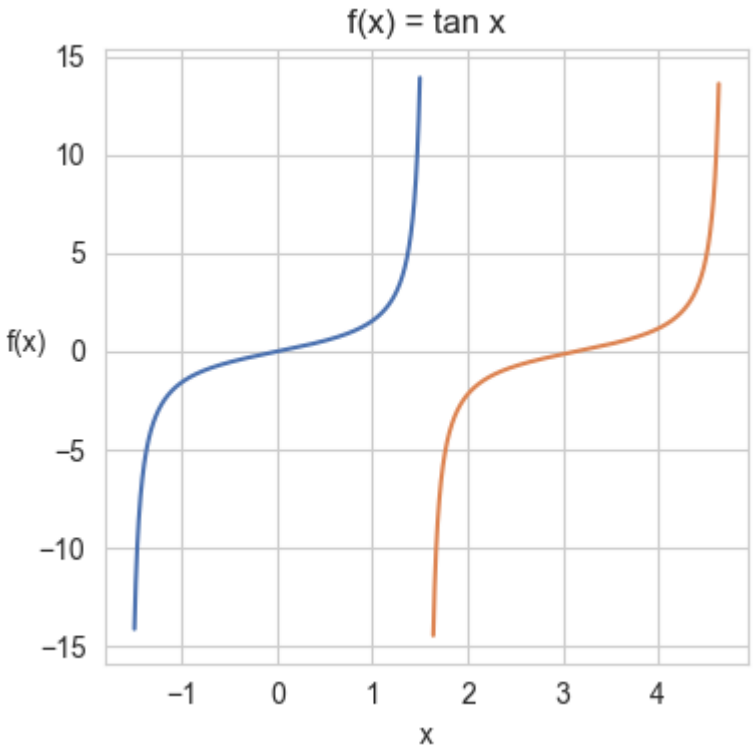
11.2 Discontinuous Function

Definition

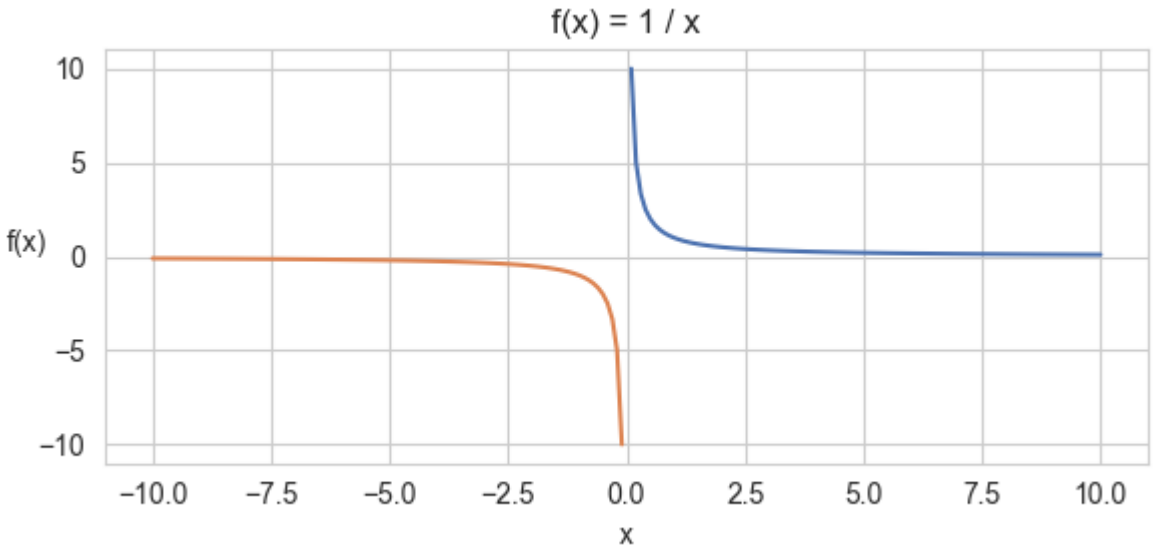
Criteria for a Discontinuous function:

$$\lim_{x \to a^-} f(x) \neq \lim_{x \to a^+} f(x)$$

Example #1



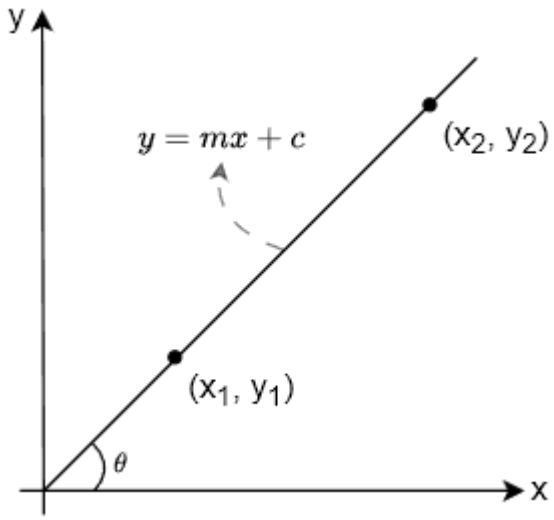
Example #2



12 Differentiation

12.1 Secant

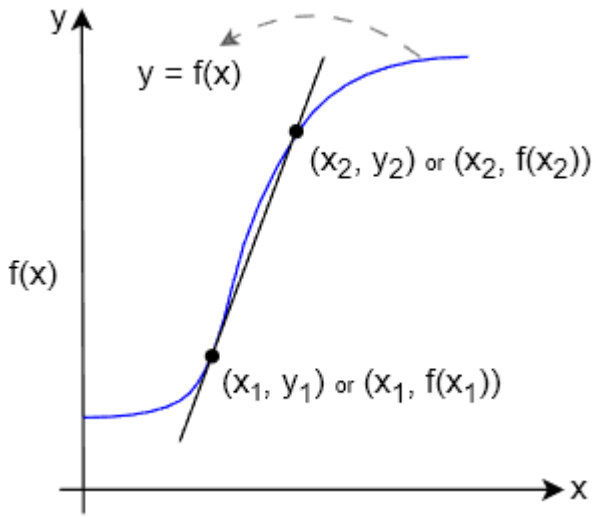
Slope of Line



Formula to calculate slope of a line intersecting two points (x_1, y_1) and (x_2, y_2)

$$m = \tan\theta = \frac{y_2 - y_1}{x_2 - x_1}$$

Slope of Secant Line

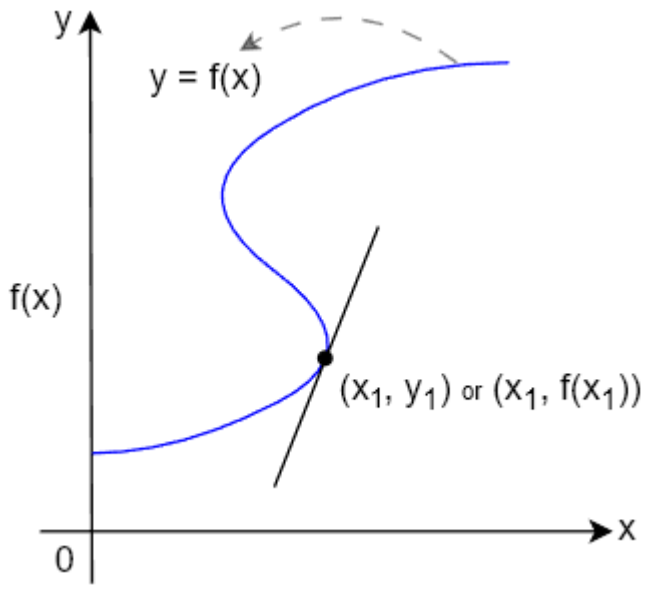


Formula for slope of a secant line intersecting a curve at two points (x_1, y_1) and (x_2, y_2)

$$m = \frac{f(x_2) - f(x_1)}{x_2 - x_1}$$

12.2 Tangent

Slope of Tangent Line (Derivative)



Formula for slope of a tangent line intersecting a curve at one single point (x, y)

$$f'(x) = \frac{d}{dx} f(x) = \lim_{\Delta x \to 0} \frac{f(x + \Delta x) - f(x)}{\Delta x}$$

12.3 Derivatives Examples

Commonly used Derivatives

(1) $\frac{d}{dx} x^n = n x^{n-1}$

(2) $\frac{d}{dx} \log(x) = \frac{1}{x}$

(3) $\frac{d}{dx} e^x = e^x$

(4) $\frac{d}{dx} c = 0$

(5) $\frac{d}{dx} \sin(x) = \cos(x)$

(6) $\frac{d}{dx} \cos(x) = -\sin(x)$

(7) $\frac{d}{dx} \tan(x) = \sec^2(x)$

(8) $\frac{d}{dx} b^x = b^x \log(b)$

12.4 Rules of Differentiation

1 Constant Rule

$$\frac{d}{dx}\Big[c \cdot f(x)\Big] = c \cdot \frac{d}{dx} f(x)$$

2 Sum Rule

$$\frac{d}{dx}\Big[f(x) + g(x)\Big] = \frac{d}{dx} f(x) + \frac{d}{dx} g(x)$$

3 Difference Rule

$$\frac{d}{dx}\Big[f(x) - g(x)\Big] = \frac{d}{dx} f(x) - \frac{d}{dx} g(x)$$

4 Product Rule

$$\frac{d}{dx}\Big[f(x) \cdot g(x)\Big] = g(x)\frac{d}{dx} f(x) + f(x)\frac{d}{dx} g(x)$$

5 Quotient Rule

$$\frac{d}{dx}\left[\frac{f(x)}{g(x)}\right] = \frac{g(x)\frac{d}{dx} f(x) - f(x)\frac{d}{dx} g(x)}{\big[g(x)\big]^2}$$

6 Chain Rule

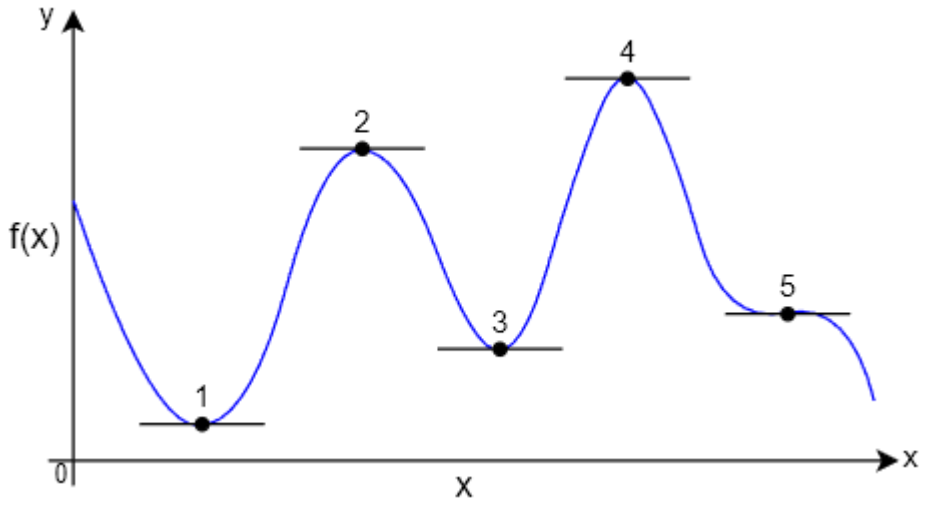
$$\frac{d}{dx} f\Big(g(x)\Big) = \frac{d}{dx} f\Big(g(x)\Big) \cdot \frac{d}{dx} g(x)$$

How Chain rule works?

$$\begin{aligned}\frac{d}{dx}\log(x^2) &= \frac{1}{x^2} \cdot 2x \\ &= \frac{2}{x}\end{aligned}$$

- Keep the inside x^2 as it is and take derivative of outside $\log$ i.e., $\frac{1}{x^2}$.
- Multiply it by the derivative of the inside x^2 i.e., $2x$.

12.5 Critical Points



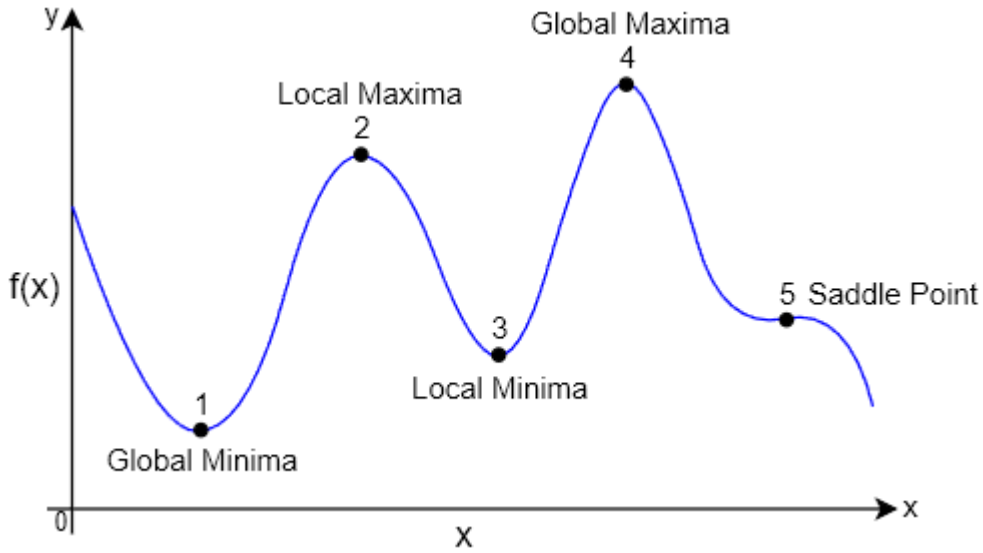
Definition

Slope of the Tangent line to a curve (i.e., derivative of a function) will be **zero at Critical Points**.

$$f'(x) = 0$$

12.6 Local Optima

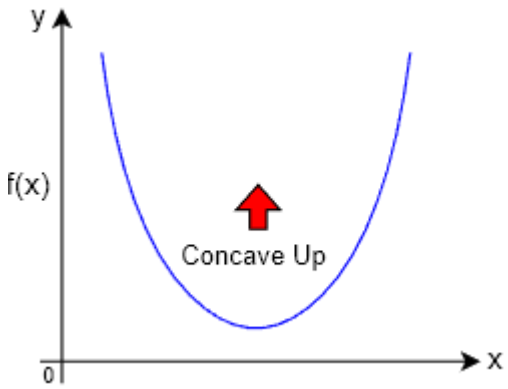
Types of Local Optima/Extrema



Two types of Local Optima/Extrema where $f'(x)$ (First derivative) is equal to zero and $f''(x)$ (Second Derivative) decides the type of Extrema:

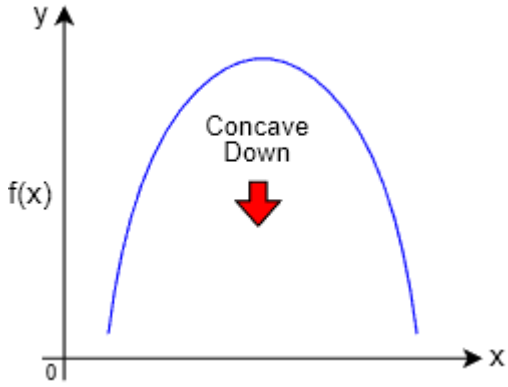
$$f'(x) = 0\,, \quad f''(x) \begin{cases} f''(x) > 0 & \text{Local Minima} \\ \\ f''(x) < 0 & \text{Local Maxima} \end{cases}$$

1 Local Minima



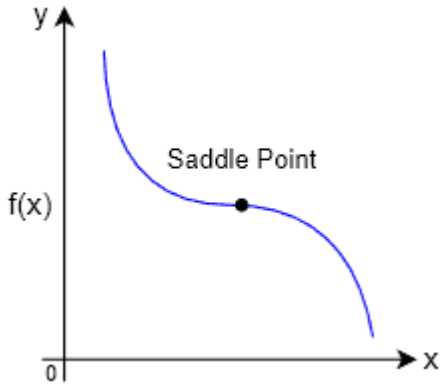
$$f''(x) > 0$$

2 Local Maxima



$$f''(x) < 0$$

Saddle Point



Non-optimal points where both $f'(x) = 0$ (First derivative) and $f''(x) = 0$ (Second derivative).

$$f''(x) = 0$$

12.7 Principles of Maxima and Minima

First and Second Derivative Test for Extrema

STEP 1: Compute First Derivative

Given function $f(x)$ compute its first derivative:

$$f'(x)$$

STEP 2: Find Critical Point

Equate the first derivative to zero to find the critical point:

$$f'(x) = 0$$

$$x = c$$

Critical Point :- Critical value of x, say c, at which slope: $f'(x) = 0$.

STEP 3: Classify Critical Point

To classify the Critical point from STEP 2 to the correct type of local Optima, compute derivative of $f'(x)$ i.e., second derivative of $f(x)$ from STEP 1:

$$f''(x)$$

Identify the type of Local Optima based on below rule:

$$f''(x) \begin{cases} \text{if} \quad f''(x) > 0: \quad \text{Local Minima} \\ \text{if} \quad f''(x) < 0: \quad \text{Local Maxima} \\ \text{if} \quad f''(x) = 0: \quad \text{Test Inconclusive} \end{cases}$$

STEP 4: Calculate Local Optima Value

Substitute the Critical Value c from STEP 2 in original function $f(x)$ to identify the actual value of Local Optima.

$$f(c) \in \mathbb{R}$$

13 Partial Differentiation

13.1 Partial Derivative

Partial Derivative of a general function

Formula to calculate Partial derivative of any general function $f(x, y)$ with two variables $\{x, y\}$

$$\frac{\partial}{\partial x} f(x, y) = \lim_{\Delta x \to 0} \frac{f(x + \Delta x,\ y) - f(x, y)}{\Delta x}$$

$$\frac{\partial}{\partial y} f(x, y) = \lim_{\Delta y \to 0} \frac{f(x,\ y + \Delta y) - f(x, y)}{\Delta y}$$

13.2 Slope of Tangent Line (Gradient)

Gradient of a function

Given Linear Binary Classifier having feature vector $\vec{x}$ of d dimension and weight vector $\vec{w}$:

$$f(w_1, w_2, \ldots, w_d, w_0) = w_1 x_1 + w_2 x_2 + \ldots + w_d x_d + w_0$$

$$f(\vec{w}, w_0) = w_1 x_1 + w_2 x_2 + \ldots + w_d x_d + w_0$$

Equation to calculate Gradient Vector for the above scalar function $f(\vec{w}, w_0)$

$$\nabla_{\vec{w}, w_0} f(\vec{w}, w_0) = \begin{bmatrix} \frac{\partial}{\partial\, w_1} f(\vec{w}, w_0) \\ \frac{\partial}{\partial\, w_2} f(\vec{w}, w_0) \\ \vdots \\ \frac{\partial}{\partial\, w_d} f(\vec{w}, w_0) \\ \frac{\partial}{\partial\, w_0} f(\vec{w}, w_0) \end{bmatrix}$$

Visualize Gradient: https://www.desmos.com/3d/e3fratrwho

14 Gradient Descent

14.1 Gradient Descent Equation

Gradient Update Rule

$$x^n = x^o \ - \ \eta \frac{\partial}{\partial x} f(x^o)$$

Where,

- x - Some random vector (not features)
- x^n - New x values after update
- x^o - Old x values before update
- η - Learning Rate
- $f(x)$ - Objective function
- $\frac{\partial}{\partial x} f(x)$ - Partial Derivative of Objective function

Gradient Update Rule (In ML Context)

$$\vec{w}^{\text{new}} = \vec{w}^{\text{old}} \ - \ \eta \frac{\partial}{\partial \vec{w}} f(\vec{w}^{\text{old}})$$

$$w_0^{\text{new}} = w_0^{\text{old}} \ - \ \eta \frac{\partial}{\partial w_0} f(w_0^{\text{old}})$$

Where,

- $\vec{w}^{\text{new}}$ - New Weight vector w after update
- $\vec{w}^{\text{old}}$ - Old Weight vector w before update
- w_0^{new} - New Bias w_0 value after update
- w_0^{old} - Old Bias w_0 value before update
- η - Learning Rate
- $f(w)$ - Objective function
- $\frac{\partial}{\partial w} f(w)$ - Partial Derivative of Objective function

14.2 Constrained Optimization

Constraint

Equation for Optimization/Minimization of Loss Function with a constraint that $\vec{w}$ is a unit vector:

$$w^*, w_0^* = \arg\min_{\vec{w}, w_0} \; l(D, \vec{w}, w_0) \qquad \text{such that: } \|\vec{w}\| = 1$$

$$= \arg\min_{\vec{w}, w_0} \; -\frac{1}{n} \sum_{i=1}^{n} \left(\frac{w^T x^{(i)} \; + \; w_0}{\|w\|} \right) y^{(i)} \qquad \text{such that: } \|\vec{w}\| = 1$$

Lagrangian Function

$$\mathcal{L}(x, \lambda) \equiv f(x) \; + \; \lambda \cdot g(x)$$

Where λ is called as **Lagrange Multiplier**.

Lagrange Multiplier for Constraint Optimization

Applying Lagrange Multiplier on Optimization equation:

$$w^*, w_0^* = \arg\min_{\vec{w}, w_0, \lambda} \; l\big(D, \vec{w}, w_0, \lambda\big)$$

$$= \arg\min_{\vec{w}, w_0, \lambda} \; -\frac{1}{n} \sum_{i=1}^{n} \Big(w^T x^{(i)} \; + \; w_0 \Big) y^{(i)} \; + \; \lambda \big(\|\vec{w}\| - 1 \big)$$

Gradient Update Rule With Lagrange Multiplier

$$\vec{w}^{t+1} = \vec{w}^{t} - \eta \left[- \sum_{i=1}^{n} x_i \cdot y_i + \lambda \frac{\vec{w}}{\|w\|} \right]$$

$$w_0^{t+1} = w_0^{t} - \eta \left[- \sum_{i=1}^{n} y_i \right]$$

Where,

- λ - is the regularizer.

15 Principal Component Analysis (PCA)

15.1 PCA Linear Equation

Linear equation for PCA:

$$\text{feature}_{\text{new}} = u_1 * \text{feature}_1 \ + \ u_2 * \text{feature}_2 \ + \ \ldots \ + \ u_n * \text{feature}_n$$

Where $\text{feature}_{\text{new}}$ is a linear combination of features: $\{\text{feature}_1, \text{feature}_2, \ \ldots, \ \text{feature}_n\}$

Principal Components

Principal Components vector $v_{n \times d}$ is calculated as:

$$\underset{n \times d}{\begin{bmatrix} x_{11} & x_{12} & \cdots & x_{1d} \\ \vdots & \vdots & \vdots & \vdots \\ x_{n1} & x_{n2} & \cdots & x_{nd} \end{bmatrix}} \cdot \underset{d \times d}{\begin{bmatrix} u_{11} & u_{12} & \cdots & u_{1d} \\ \vdots & \vdots & \vdots & \vdots \\ u_{d1} & u_{d2} & \cdots & u_{dd} \end{bmatrix}} = \underset{n \times d}{\begin{bmatrix} v_{11} & v_{12} & \cdots & v_{1d} \\ \vdots & \vdots & \vdots & \vdots \\ v_{n1} & v_{n2} & \cdots & v_{nd} \end{bmatrix}}$$

$$X^T_{n \times d} \cdot u_{d \times d} = v_{n \times d}$$

15.2 Optimization of Variance in Projection

Equation for Optimization/maximizing variance during projection of vectors:

$$\vec{u}^* = \arg\max_{\vec{u}} \ \frac{1}{n} \sum_{i=1}^{n} \left(\frac{\vec{x_i}^T \cdot u_i}{\|u\|} \ - \ \mu \right)^2$$

15.3 Constraint Optimization

Constraint

Equation for maximizing variance during projection of vectors with constraint enforced on it:

$$\vec{u}^* = \arg\max_{\vec{u}} \ \frac{1}{n} \sum_{i=1}^{n} \left(\frac{\vec{x_i}^T \cdot \vec{u_i}}{\|\vec{u}\|} \ - \ \mu \right)^2 \quad \text{such that: } \|\vec{u}\| = 1$$

Equation for maximizing variance during projection of vectors with constraint optimization:

$$\vec{u}^* = \arg\max_{\vec{u}} \ \frac{1}{n} \sum_{i=1}^{n} \left(\vec{x_i}^T \cdot \vec{u_i} \right)^2 \ + \ \lambda\big(\|\vec{u}\| - 1\big)$$